

Dynamic Virtual Network Embedding Algorithm Based on Graph Convolution Neural Network and Reinforcement Learning

Peiying Zhang[✉], Chao Wang[✉], Neeraj Kumar[✉], *Senior Member, IEEE*, Weishan Zhang[✉], and Lei Liu

Abstract—Network virtualization (NV) is a technology with broad application prospects. Virtual network embedding (VNE) is the core orientation of VN, which aims to provide more flexible underlying physical resource allocation for user function requests. The classical VNE problem is usually solved by the heuristic method, but this method often limits the flexibility of the algorithm and ignores the time limit. In addition, the partition autonomy of physical domain and the dynamic characteristics of virtual network request (VNR) also increase the difficulty of VNE. This article proposed a new type of VNE algorithm, which applied reinforcement learning (RL) and graph neural network (GNN) theory to the algorithm, especially the combination of graph convolutional neural network (GCNN) and RL algorithm. Based on a self-defined fitness matrix and fitness value, we set up the objective function of the algorithm implementation, realized an efficient dynamic VNE algorithm, and effectively reduced the degree of resource fragmentation. Finally, we used comparison algorithms to evaluate the proposed method. Simulation experiments verified that the dynamic VNE algorithm based on RL and GCNN has good basic VNE characteristics. By changing the resource attributes of physical network and virtual network, it can be proved that the algorithm has good flexibility.

Index Terms—Fitness matrix, graph convolution neural network, reinforcement learning (RL), virtual network embedding (VNE).

Manuscript received June 8, 2021; accepted June 29, 2021. Date of publication July 6, 2021; date of current version June 7, 2022. This work was supported in part by the National Key Research and Development Program of China under Grant 2018YFE0116700; in part by the National Natural Science Foundation of China under Grant 62001357 and Grant 62072469; in part by the Major Scientific and Technological Projects of CNPC under Grant ZD2019-183-006; in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2020A1515110496 and Grant 2020A1515110079; and in part by the Shandong Provincial Natural Science Foundation, China, under Grant ZR2020MF006 and Grant ZR2019MF049. (Corresponding authors: Neeraj Kumar; Peiying Zhang; Weishan Zhang.)

Peiying Zhang, Chao Wang, and Weishan Zhang are with the College of Computer Science and Technology, China University of Petroleum (East China), Qingdao 266580, China (e-mail: zhangpeiying@upc.edu.cn; wangch_upc@qq.com; zhangws@upc.edu.cn).

Neeraj Kumar is with the Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology, Deemed to be University, Patiala 147004, India, also with the Department of Computer Science and Information Engineering, Asia University, Taichung 41354, Taiwan, and also with the School of Computer Science, University of Petroleum and Energy Studies, Dehradun 248007, India (e-mail: neeraj.kumar@thapar.edu).

Lei Liu is with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710071, China (e-mail: leiliu@xidian.edu.cn).

Digital Object Identifier 10.1109/JIOT.2021.3095094

I. INTRODUCTION

THE FACT that the traditional Internet architecture is becoming rigid has promoted the development of future network technologies [1]. Network virtualization (NV) will realize the effective management of the next-generation Internet and become an important part of the future network architecture [2], [3]. NV mainly realizes the decoupling of network services, functions, and specific underlying hardware, which greatly improves the flexibility and scalability of the underlying network architecture. NV involves two main roles: 1) infrastructure provider (InP) and 2) service provider (SP) [4]. The process that the two cooperate to allocate physical network resources for virtual network requests (VNRs) is called virtual network embedding (VNE). The ideal result of VNE is to reasonably schedule the underlying network resources to receive as many VNRs as possible, improve the acceptance rate of virtual network, realize load balancing, maximize resource consumption revenue and reduce costs [5], [6].

VNE is a typical NP-hard problem, which determines that the VNE problem can only obtain an approximate optimal solution instead of an optimal solution [7], [8]. Usually, the heuristic method is used to solve the VNE process, or VNE is modeled as a combinatorial optimization problem, and then (integer) linear programming is used to solve it. The above methods are traditional solutions to the VNE problem, which play a positive role in a certain social application and technical background. The analysis shows that there are several common disadvantages in traditional VNE solutions.

- 1) The constraints and optimization objectives of VNE are made manually. Unnatural conditional control will damage the robustness and flexibility of the VNE algorithm.
- 2) The VNE algorithm with differentiated Quality of Service (QoS) may have the high real-time requirement, so it is time sensitive. Existing algorithms usually ignore this problem.
- 3) Ignore the dynamic characteristics of VNE, i.e., during the VNR cycle, the resource configuration of virtual nodes and links, the virtual topological structure, and the number of physical network resources are all in dynamic changes.

The application of machine learning (ML) algorithms in VNE has achieved great success [9], [10]. Through the efficient interaction between learning agent and environment,

reinforcement learning (RL) has excellent decision-making capabilities. At each time step t , the intelligent agent interacts with the environment to obtain the environment state s_t . After that, the agent will apply an action a_t to the environment to transfer the state of the environment to s_{t+1} , and at the same time, the environment will feedback to the agent a reward r_t . The goal of RL is to help the agent maximize the reward signal. The detailed application of RL will be introduced below. The VNE algorithm based on RL has been well experimented, so RL can be applied to our work.

Problems such as node ordering and pattern matching can all be simplified by graph-related theories and techniques. Graph neural network (GNN) is a new type of ML model architecture that can aggregate graph features (degrees, distance to specific nodes, node connectivity, etc.) on nodes [11]. The model can be used to cluster nodes and links according to the physical nodes and physical links attribute characteristics (CPU, storage, bandwidth, delay, etc.) and search for the best VNE strategy by intelligent agent training. Graph convolution neural network (GCNN) is a natural generalization form of GNN, which is highly suitable for graph structures of any topological form [12]. Therefore, the organic combination of VNE and GCNN has a good prerequisite. To this end, the main work of this article is as follows.

- 1) A VNE algorithm based on RL and GCNN is proposed. In particular, GCNN is used to automatically extract the features of underlying network, which optimizes the selection of VNE decision.
- 2) We notice the dynamic characteristics of VNE process, and adopt a dynamic VNE method based on fitness value to deal with the changing virtual network topology and resource attributes.
- 3) We conducted a large number of simulation experiments and compared the typical indicators of VNE with other algorithms to verify the effectiveness of the algorithm. In addition, the flexibility of the algorithm is verified by changing the attribute parameter distribution of physical network and VNRs.

The remainder of this article is arranged as follows. Section II introduces the related work of VNE. Section III gives the necessary network model to solve the VNE problem and describes the related technical problems. Section IV formulates the related problems of VNE. Section V introduces the design and implementation details of the algorithm. Section VI verifies and analyzes the performance of the algorithm through simulation experiments. In the last section, we summarize this whole article.

II. RELATED WORK

The heuristic method is the traditional method for solving the VNE process, including the metaheuristic algorithm, mathematical optimization, and so on. VNE solutions based on ML often use algorithms or models, such as deep learning (DL), RL or graph theory, which has gradually become the mainstream method to solve NP-hard problems, such as VNE [13].

A. VNE Solution Based on Heuristic

The more classic is the node-link two-stage coordinated and embedded ViNEYard algorithm proposed by Chowdhury *et al.* [14]. This algorithm modeled VNE as a mixed-integer linear programming problem, and then implemented D-ViNE and R-ViNE using deterministic and random methods, respectively. The results showed that these algorithms can effectively improve the revenue of VNE. Increasing the VNE revenue or reducing the embedding cost is an important goal of the VNE algorithm. Based on the Markov random walk model theory, Cheng *et al.* [15] sorted the importance of network nodes according to network topology attributes, and proposed two representative VNE algorithms based on node sorting. According to whether the virtual link was divided into paths, it was decided to use the multicommodity flow algorithm or the breadth-first search algorithm to perform the virtual link mapping process. These two algorithms had effectively improved the long-term average revenue, and the method based on node sorting has also become a typical and popular method in VNE. Zhang *et al.* [16] first considered the VNE algorithm with multidimensional resource constraints of computing, network and storage, and proposed two heuristic algorithms NRM-VNE and RCR-VNE as baseline algorithms.

Since VNE can be modeled as a search problem in a larger 3-D space, the metaheuristic method can be used as an important means to solve VNE. The metaheuristic algorithm is an improvement of heuristic algorithm, which adds local search on the basis of heuristic algorithm [17]. Particle swarm algorithm (PSO) is a metaheuristic algorithm commonly used to solve VNE [18]–[20]. Different VNE strategies can be used as particles to perform a global optimal search based on the objective function set in advance.

As we mentioned earlier, the heuristic method of manually formulating constraints and features for the VNE algorithm greatly limits the flexibility of the VNE algorithm. Moreover, with the rapid expansion of physical network and user scale, the global search strategy efficiency of heuristic algorithm gradually decreases, and there is a big gap between the time performance and the latest VNE algorithm.

B. VNE Solution Based on ML

With the successful practice of ML algorithms in real life, scholars began to explore and tried to apply ML algorithms in VNE, among which the RL algorithm was the most representative [21]. Haeri and Trajkovic [22] modeled VNE as a Markov decision process, and then used the Monte Carlo search tree (MCST) algorithm to implement the node mapping process. In the stage of virtual link embedding, the authors used the multicommodity flow algorithm to implement the MaVen-M algorithm, and the shortest path algorithm to implement the MaVen-S algorithm. The profitability and profitability of InP illustrated the effectiveness of the algorithm. It cannot be ignored that the MCST algorithm should perform a complete VNE process every time the virtual nodes are embedded, which makes the algorithm's time cost higher. Q -learning is another commonly used RL model. Yuan *et al.* [23] implemented the VNE algorithm based on this model to find the

optimal VNE strategy for RL agent, in which the Q -matrix and the Q -table played an important role. One problem with this algorithm is that the extraction of virtual network features is done manually. The efficiency of the algorithm will decrease when the VNR scale is large or the structure is complex. Yao *et al.* [24] implemented a VNE algorithm based on single-layer policy network. According to the historical embedded data of virtual network, RL agent used strategy gradient descent to find the optimal embedding strategy.

Dolati *et al.* [25] and Wang *et al.* [26] also combined DL and RL, and their deep reinforcement learning (DRL)-based VNE algorithm also provided new ideas for solving this problem.

In the field of ML, the VNE algorithm based on graph theory has gradually come into people's view. Habibi *et al.* [27] creatively combined GNN and VNE, and then proposed the GraphViNE algorithm based on graph automatic encoder. The biggest feature of this algorithm is that servers with similar resources are clustered, which ultimately effectively reduces the running time of the algorithm. Yan *et al.* [28] paid attention to the time constraint of VNE and applied GNN, especially GCNN, to VNE. It was worth noting that the authors proposed a parallel framework and multiobjective reward function based on DRL, which took into account VNE revenue, acceptance rate, and load balance. Zhao and Parhami [29] and Ghazar and Samaan [30], respectively, proposed the VNE algorithm based on graph feature space alignment and VNE algorithm based on subgraph matching. The former mainly associated physical nodes with virtual nodes, while the latter utilized a hierarchical physical management architecture. Both of them are good applications of graph theory in the VNE problem.

The above research does not support the embedding of dynamic virtual networks well, which will greatly affect the utilization of resources. Dehury and Sahoo [31] focused on the dynamic characteristics of virtual network topology, and proposed a dynamic VNE algorithm based on fitness value. The function of fitness value was to mark the utilization degree of physical resources. At the same time, the authors used the method of studying part of physical servers to replace the method of studying all physical servers, which effectively improved the efficiency of the algorithm. In addition, [32] and [33] are also excellent representatives of the dynamic VNE algorithm. But the analysis finds that they do not fully apply ML methods.

III. NETWORK MODEL AND PROBLEM DESCRIPTION

For easy viewing, we summarize the commonly used network representation symbols in Table I.

A. Physical Network and VNR Modeling

Physical network can be regarded as a weighted undirected graph $G^P = \{N^P, L^P, A^N, A^L\}$. N^P represents the node set composed of all underlying physical nodes, and L^P represents the link set composed of all physical links. A^N and A^L represent physical node attribute set and physical link attribute set, respectively. We mainly regard CPU resources

TABLE I
NOTATION OF NETWORK MODEL

Notation	Description
G^P	physical network
N^P	physical node
L^P	physical link
A^N	physical node resource attributes
A^L	physical link resource attributes
G_i^V	the i -th virtual network request
N^V	virtual node
L^V	virtual link
R^V	virtual node resource requirements
R^L	virtual link resource requirements

as physical node attributes and bandwidth resources as physical link attributes, namely, $A^N = \{c_{n_1^p}, c_{n_2^p}, \dots, c_{n_{|N^P|}^p}\}$ and $A^L = \{b_{l_1^p}, b_{l_2^p}, \dots, b_{l_{|L^P|}^p}\}$, where $|N^P|$ and $|L^P|$ represent the total number of physical nodes and physical links, respectively.

Similarly, virtual network is regarded as a weighted undirected graph $G_i^V = \{N^V, L^V, R^N, R^L\}$, and G_i^V indicates that this is the i th virtual network. N^V represents the set of all virtual nodes, and L^V represents the set of all virtual links. R^N and R^L , respectively, represent the resource requirement attribute set of virtual node and virtual link, where $N^V = \{c_{n_1^v}, c_{n_2^v}, \dots, c_{n_{|N^V|}^v}\}$ represents the CPU resource requirement of virtual node, and $L^V = \{b_{l_1^v}, b_{l_2^v}, \dots, b_{l_{|L^V|}^v}\}$ represents the bandwidth resource requirement of virtual link. $|N^V|$ and $|L^V|$, respectively, represent the number of virtual nodes and virtual links in the virtual network.

B. VNE Model and Description

Based on the physical network and virtual network model, the VNR can be expressed as $VNR_i = (G_i^V, t_a, t_e)$, where t_a represents the time when the VNR arrives, and t_e represents the time when the VNR leaves. The VNE process can be modeled as $G_i^V(N^V, L^V) \uparrow G^P(N^P, L^P)$, where $G^P(N^P, L^P)$ represents a partial subgraph of physical network. Thus, a complete VNE process includes two stages: 1) node mapping and 2) link mapping. Fig. 1 gives a visual representation of the network topology, and shows the result of two VNRs embedded in the physical network. We mark the CPU resource demand and bandwidth resource demand, respectively, next to the network node and link. The target node and link should have sufficient resources to carry them.

C. Dynamic VNE

Changes in the number of network users or fluctuations in workload will cause the structural change of the virtual network. The most intuitive impact of changes in the number of users is the changes in virtual nodes and links, i.e., changes in structural configuration. Workload fluctuations usually change the number of resource requirements of virtual nodes or links, i.e., the resource configuration changes. The

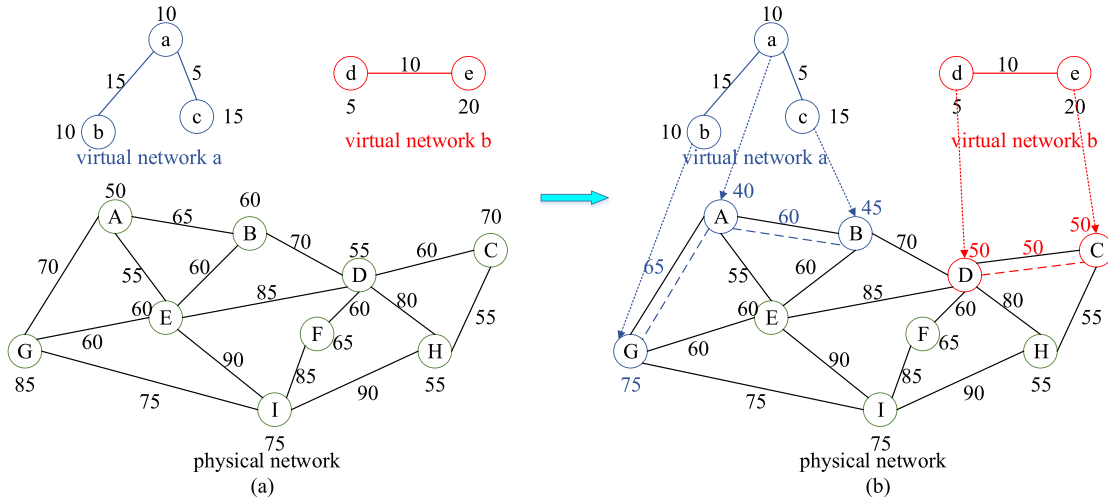


Fig. 1. Schematic of physical network and virtual network. (a) VNRs before embedding. (b) Possible result of VNRs embedding.

above changes are real in the process of VNE. The virtual network whose structure configuration or resource configuration changes with time interval is called dynamic virtual network, and the process of mapping dynamic virtual network to physical network is called dynamic VNE.

Fig. 2 shows the change process of virtual network structure configuration and resource configuration in different time intervals. $t = 0$ to $t = 5$ is a continuous VNE process. When $t = 0$, the virtual network structure is the initial form, and the resource requirements of virtual nodes and links are shown in the figure. When $t = 1$, the configuration of virtual network is changed, and virtual node d and its related virtual links $l^v(a, d)$, $l^v(b, d)$, and $l^v(c, d)$ are deleted. When $t = 2$, the structural configuration change process of virtual network ends, and a new virtual network topology with three virtual nodes and two virtual links is formed. When $t = 3$, the configuration of virtual network changes again, and a new virtual node e and a virtual link $l^v(c, e)$ are added. When $t = 4$, the resource configuration of virtual network has changed. The number of CPU resource requirements for node a increases from 10 to 15 units, and the number of CPU resource requirements for node b decreases from 15 to 10 units. When $t = 5$, the number of bandwidth resources of virtual link $l^v(a, b)$ also changes, from the original 5 to 20 units.

IV. PROBLEM FORMULATION

A. Restrictions

We use the binary variable $\varepsilon_{n_k^v}^{n^p}$ to measure whether the physical node n^p is embedded by the virtual node n_k^v , where k is the virtual node label, $k = 1, 2, \dots, |N_i^V|$. $|N_i^V|$ represents the number of virtual nodes in virtual network G_i^V . Then $\varepsilon_{n_k^v}^{n^p}$ can be expressed as follows:

$$\varepsilon_{n_k^v}^{n^p} = \begin{cases} 1, & \text{if } n_k^v \uparrow n^p \\ 0, & \text{others.} \end{cases} \quad (1)$$

Another binary variable $\delta_{l_k^v}^{l^p}$ is used to measure whether the virtual link l_k^v is mapped to the physical link l^p , where k is the virtual link label, $k = 1, 2, \dots, |L_i^V|$. $|L_i^V|$ indicates the

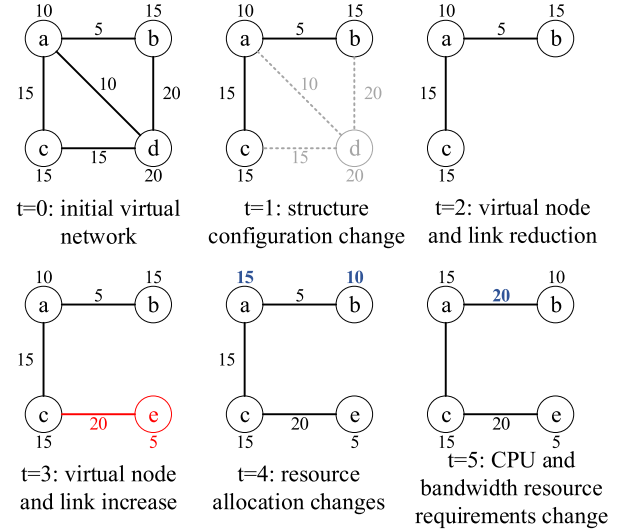


Fig. 2. Dynamic virtual network example.

number of virtual links in virtual network G_i^V . Then $\delta_{l_k^v}^{l^p}$ can be expressed as follows:

$$\delta_{l_k^v}^{l^p} = \begin{cases} 1, & \text{if } l_k^v \uparrow l^p \\ 0, & \text{others.} \end{cases} \quad (2)$$

VNE needs to follow resource, location and access control constraints. Here, we mainly set the resource constraints of VNE. The specific constraint content is shown in the following formula:

$$\sum_{k=1}^{|N_i^V|} \varepsilon_{n_k^v}^{n^p} = 1, n_k^v \in G_i^V \quad (3)$$

$$\sum_{k=1}^{|L_i^V|} \delta_{l_k^v}^{l^p} \geq 1, l_k^v \in G_i^V \quad (4)$$

$$A_{-}c_n^p = c_{n^p} - \sum_{k=1}^{|N_i^V|} \varepsilon_{n_k^v}^{n^p} \cdot c_{n_k^v}, c_{n_k^v} \text{ from different } G^V \quad (5)$$

$$A_{b_l^p} = b_{l^p} - \sum_{k=1} \delta_{l_k^p} \cdot b_{l_k^p}, b_{l_k^p} \in G_i^V, i = 1, 2, \dots, |\text{VNR}| \quad (6)$$

$$c_{n_k^v} \leq A_{c_n^p}, \text{ if } \varepsilon_{n_k^v}^p = 1 \quad (7)$$

$$b_{l_k^p} \leq A_{b_l^p}, \text{ if } \delta_{l_k^p}^p = 1. \quad (8)$$

Formula (3) shows that virtual nodes n_k^v from the same virtual network G_i^V can only be mapped to different physical nodes. Formula (4) shows that a virtual link l_k^v may be mapped to multiple physical links. Formulas (5) and (6) represent the current available resource capacities of physical nodes n^p and links l^p , respectively. Formula (7) and (8) indicate that when virtual node n_k^v and virtual link l_k^v are embedded on physical node n^p and physical link l^p , respectively, the resource capacity of n^p and l^p is sufficient.

B. Evaluation Index

The classical evaluation indexes of the VNE algorithm include long-term average revenue, resource consumption cost, resource utilization rate, virtual network acceptance rate, and revenue-cost ratio. Among them, the revenue of VNE are measured by the consumption of CPU resources and bandwidth resources

$$R(G_i^V) = \sum_{n^v \in N^V} c_{n^v} + \sum_{l^v \in L^V} b_{l^v}, N^V, L^V \in G_i^V \quad (9)$$

where c_{n^v} represents the CPU resource requirement of virtual node n^v and b_{l^v} represents the bandwidth resource requirement of virtual link l^v .

The resource consumption cost of VNE is expressed as follows:

$$C(G_i^V) = \sum_{n^v \in N^V} c_{n^v} + \sum_{l^v \in L^V} \sum_{l^p \in L^P} b_{l^p}^p, N^V, L^V \in G_i^V, L^P \in G^P. \quad (10)$$

The main difference between the two is reflected in the consumption of bandwidth resources. Since a virtual link may be mapped to multiple physical links due to path segmentation, the cost of bandwidth consumption will increase, and the revenue of bandwidth consumption only depends on its own bandwidth resource demand. Based on both, the long-term average revenue of VNE is defined as follows:

$$R = \lim_{T \rightarrow \infty} \frac{\sum_{t=0}^T \sum_{i=1}^{|\text{VNR}|} R(G_i^V, t)}{T} \quad (11)$$

where $\sum_{t=0}^T \sum_{i=1}^{|\text{VNR}|} R(G_i^V, t)$ refers to the total revenue of VNR embedding in the time range T .

The VNE revenue-cost ratio is defined as follows:

$$R/C = \lim_{T \rightarrow \infty} \frac{\sum_{t=0}^T \sum_{i=1}^{|\text{VNR}|} R(G_i^V, t)}{\sum_{t=0}^T \sum_{i=1}^{|\text{VNR}|} C(G_i^V, t)} \quad (12)$$

where $\sum_{t=0}^T \sum_{i=1}^{|\text{VNR}|} C(G_i^V, t)$ refers to the resource cost consumed by the VNR embedding in the time range T .

The VNR acceptance rate is defined as follows:

$$\text{AR} = \lim_{T \rightarrow \infty} \frac{\sum_{t=0}^T |\text{VNR}|_{\text{acc}}}{\sum_{t=0}^T |\text{VNR}|_{\text{arr}}} \quad (13)$$

where $|\text{VNR}|_{\text{acc}}$ is the number of VNRs successfully mapped to the physical network in the VNE process. $|\text{VNR}|_{\text{arr}}$ is the total number of VNRs that come in the VNE process.

C. Fitness Matrix and Value

There needs to be a standard to record the degree of utilization of CPU resources, and the fitness matrix can play a role. The value in the matrix is the remaining CPU capacity, and the calculation method is shown in (5). We use $f_{ij}(t) = A_{c_n^p}$ to mark every element in the fitness matrix, where i represents the physical node in each row, and j represents the virtual node in each column. Therefore, the dimension of fitness matrix depends on the number of virtual nodes embedded at time t . The sign $F(t) = f_{ij}(t)$ is used to refer to the fitness matrix. We specify that the elements of fitness matrix are recorded as follows:

$$f_{ij}(t) = \begin{cases} \infty, & f_{ij} \leq 0 \\ A_{c_n^p}, & \text{others.} \end{cases} \quad (14)$$

We define the concept of fitness value as the availability of physical nodes to characteristic virtual nodes. The calculation method is as follows:

$$\alpha_j(t) = \sum_{i=1}^{|\text{N}^p|} \varepsilon_{n_j^v}^p \cdot \frac{f_{ij}(t)}{c_{n^p}}. \quad (15)$$

We use an adjoint matrix of a fitness matrix to mark whether there is a link between any two physical nodes. The rows and columns of the adjoint matrix represent the physical node numbers, and the value of each element in the matrix is defined as follows:

$$\text{Adj}(t) = \begin{cases} b_{l^p}, & l^p \text{ between } n_i^p \text{ and } n_j^p \\ \infty, & \text{no link between } n_i^p \text{ and } n_j^p. \end{cases} \quad (16)$$

Considering CPU resources and bandwidth resources, the overall fitness value of the algorithm is calculated as follows:

$$\beta(t) = \sum_{j=1}^{\text{num}_{n^v}} \alpha_j(t) + \sum_{i=1}^{|\text{L}^p|} \sum_{j=1}^{\text{num}_{l^v}} \delta_{l_j^p}^p \cdot \frac{A_{b_{l^p}}}{b_{l^p}} \quad (17)$$

where num_{n^v} and num_{l^v} , respectively, represent the total number of virtual nodes and virtual links successfully mapped in the entire VNR process.

The ultimate goal of the algorithm is

$$\begin{aligned} & \text{Minimize} \sum_{j=1}^{\text{num}_{n^v}} \min_{1 \leq i \leq |\text{L}^p|} \left(\varepsilon_{n_j^v}^p \cdot \frac{A_{c_{n^p}}}{c_{n^p}} + \delta_{l_j^p}^p \cdot \frac{A_{b_{l^p}}}{b_{l^p}} \right) \\ & = \text{Minimize} \sum_{j=1}^{\text{num}_{n^v}} \min_{1 \leq i \leq |\text{L}^p|} \left(\varepsilon_{n_j^v}^p \cdot \frac{f_{ij}(t)}{c_{n^p}} + \delta_{l_j^p}^p \cdot \frac{A_{b_{l^p}}}{b_{l^p}} \right) \\ & = \text{Minimize} \beta(t) \end{aligned} \quad (18)$$

where $\varepsilon_{n_j^v}^p$ is used to measure whether the virtual node n_j^v is mapped to the physical node n_i^p , and $\delta_{l_j^p}^p$ is used to measure whether the virtual link l_j^v is mapped to the physical link l_i^p .

V. ALGORITHM DESIGN AND IMPLEMENTATION

RL mainly includes four entities: 1) intelligent agent; 2) state s_t ; 3) action a_t ; and 4) reward r_t . When RL is applied to the VNE problem, we use a self-defined GCNN model as RL agent, which can extract network features for training, and then get the VNE strategy. The specific content of the GCNN model will be introduced in the following. s_t refers to the information that agent can obtain from virtual network environment. We define s_t to include the available CPU resources of each physical node, the available bandwidth resources of each physical link, the resource requirements of each virtual node and each virtual link, the virtual nodes and virtual links that have been successfully mapped to the physical network in the current VNR, and the virtual nodes and virtual links that have not been mapped in the current VNR. a_t refers to a VNE strategy adopted by agent, which includes node mapping strategy and link mapping strategy. r_t is a reward function that agent gets by adopting some embedding strategy to the environment. It should be noted that RL agent has no strict objective function or label. It accumulates reward signals through continuous training. We define the calculation method of r_t obtained by a_t as follows:

$$r(a_t) = R(G_i^V) \cdot R/C. \quad (19)$$

Convolutional neural network (CNN) can automatically extract the advanced features of images. It has been shown that CNN can provide a great help in dealing with VNE-related problems [21], [24]. But the original intention of CNN design is to solve the Euclidean graph with elements arranged in order, and the efficiency will be much lower when dealing with any graph structure such as network topology. Defferrard *et al.* [34] proposed a method of convolution operation on random graph topology under the background of spectrograph theory, i.e., the GCNN method. The essence of GCNN is the Fourier transform. The key to the problem is to find a Laplacian in the graph and use an orthogonal factor to characterize the network topology. The following Fourier transform form can be used:

$$F(\lambda_v) = \hat{f}(\lambda_v) = \sum_{i=1}^N f(i) \mu_v^*(i) \quad (20)$$

where f represents the vector, N represents the dimension of the vector, $f(i)$ corresponds to the network node, and $\mu_v^*(i)$ represents the i th component of the v th feature vector. The Fourier change of f is the inner product operation of the feature vector $\mu_v^*(i)$ corresponding to the feature value. Namely

$$\hat{f} = U^T f. \quad (21)$$

Perform the inverse Fourier transform to get

$$f = U^T \hat{f}. \quad (22)$$

According to convolution theorem, the Fourier transform of function convolution is the product of function Fourier transform, then the Fourier transform of f and convolution kernel g on the network topology diagram is defined as follows:

$$f * g = \Gamma^{-1}(\hat{f}(\omega) \cdot \hat{g}(\omega)) = \frac{1}{2\pi} \int \hat{f}(\omega) \cdot \hat{g}(\omega) e^{-i\omega t} d\omega \quad (23)$$

Algorithm 1 Dynamic VNE Algorithm

Input: $G^P(N^P, L^P)$, $G^V(N^V, L^V)$, D ;

- 1: Calculate fitness value by formulas (15) and (16);
- 2: $flag = FALSE$;
- 3: **while** $flag = FALSE$ **do**
- 4: $flag = TRUE$;
- 5: **for** each $n_i^v \in N^V \& n_j^p \in N^P$ **do**
- 6: $n_i^v \uparrow n_j^p$;
- 7: **if** another $n_k^v \uparrow n_j^p$ **then**
- 8: $n^v = n_i^v + n_k^v$;
- 9: Calculate new fitness value of n^v ;
- 10: **else**
- 11: Calculate new fitness value of n_k^v ;
- 12: **end if**
- 13: **if** $\beta(t) \geq \beta(t)'$ **then**
- 14: $flag = TRUE$;
- 15: **end if**
- 16: **end for**
- 17: **end while**
- 18: **while** $D \geq 0$ **do**
- 19: $D = D - 1$;
- 20: Perform a virtual network embedding process;
- 21: **end while**

where $g = \sum_{k=0}^K \alpha_k \Lambda^k$, α_k is the training parameter and Λ is the diagonal matrix with eigenvalue λ_v . Therefore, the final output of GCNN is as follows:

$$g_{-o} = \sum_{k=0}^K \alpha_k \lambda_v f. \quad (24)$$

We organize the topological features extracted by GCNN into a column vector, and make it output a probability distribution equal to the number of physical network nodes through the softmax operation. Then we use the asynchronous advantage actor-critic algorithm to optimize the parameters of neural network. The algorithm includes an actor network and a critic network. The former is used to generate a set of parameterized strategies π^θ , and the latter is used to generate a set of parameterized estimated values $v^{\pi^\theta}(s_t, \theta_v)$. Considering the important processes of feature extraction, strategy generation, and model training, the GCNN model based on RL is shown in Fig. 3.

The execution of dynamic VNE requires a predetermined duration D of the entire VNE. The process of dynamic VNE is shown in Algorithm 1.

The asynchronous advantage actor-critic algorithm is the key process of realizing RL agent training. This algorithm uses a model parallel training method, which can effectively improve the execution efficiency and convergence of the algorithm. The training process of the asynchronous advantage actor-critic algorithm is shown in Algorithm 2.

VI. PERFORMANCE EVALUATION AND ANALYSIS

A. Parameter Setting

Our simulation experiment is carried out in Anaconda3 + PyCharm3.6. We programmatically generate a physical

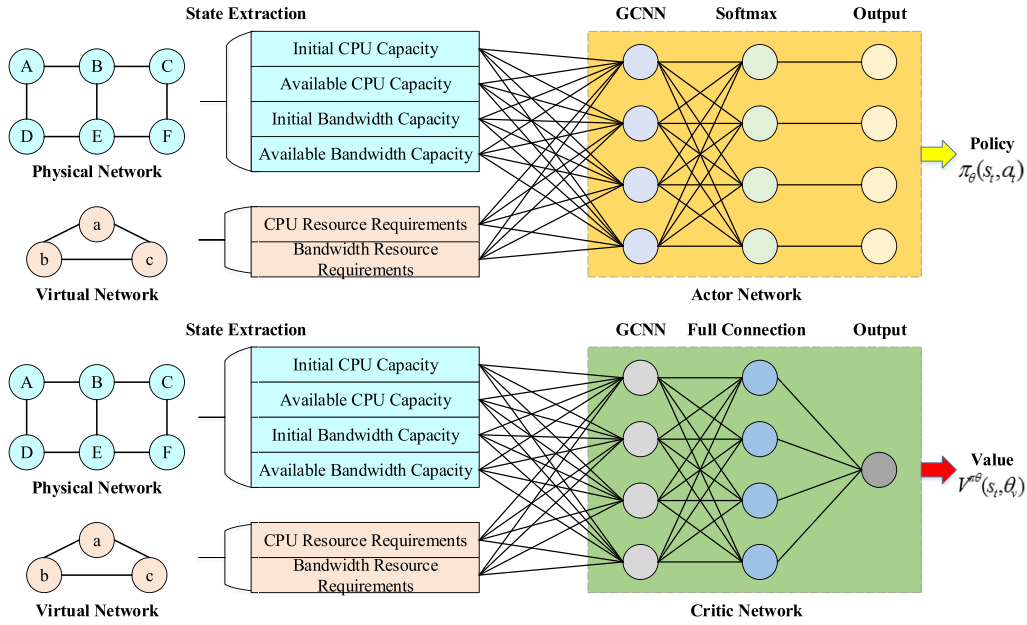


Fig. 3. GCNN model based on RL.

Algorithm 2 Training Process

```

1: Initialize parameters of actor – critic network;
2: while TRUE do
3:   Update parameters of actor – critic network;
4:   Learning new strategies from environment;
5:   Update  $s_t, a_t, r_t, s_{t+1}$ ;
6: end while
7: for  $i \in n$  do
8:   Initialize agent  $a[i]$ ;
9: end for
10: while TRUE do
11:   for  $i \in n$  do
12:     Training in environment;
13:   end for
14:   Update parameters of actor – critic network;
15:   for  $i \in n$  do
16:     Update parameters of  $a[i]$ ;
17:   end for
18: end while

```

network and 1000 virtual networks, and save them in *.txt* document. The physical network consists of 100 nodes and about 600 links. The CPU resource capacity and bandwidth resource capacity are randomly and evenly distributed between 50 and 100 units. 1000 virtual networks simulate the Poisson process to reach the physical network, forming a continuous VNR process. Each virtual network randomly contains 2 to 12 virtual nodes, and each node has a 50% probability to connect with each other. CPU resource demand and bandwidth resource demand of virtual nodes and virtual links are distributed randomly and evenly among 1–50 units. Specifically, the virtual network can reach 5 times in 100-time units, and the life cycle of each VNR is exponential distribution. The main parameters used in the simulation experiment are shown in Table II.

TABLE II
PARAMETER SETTING

Parameter	Value
Physical nodes	100
Physical links	600
CPU capacity	U[50,100]
Bandwidth capacity	U[50,100]
Virtual networks	1,000
Virtual nodes	U[2,12]
CPU requirements	U[1,50]
Bandwidth requirements	U[1,50]

B. Comparison Algorithms and Evaluation Indexes

We select a heuristic VNE algorithm NodeRank [15], an RL-based VNE algorithm MCST-VNE [22] and a GCNN-based VNE algorithm GCN-VNE [28] as comparison algorithms. Then the performance of the algorithms is compared from two aspects: 1) long-term average revenue of VNE and 2) acceptance rate of VNR. The four algorithms, including our algorithm, are described and summarized in Table III.

In addition, we try to explore the impact of changing the resource capacity of the physical network and the resource demand of VNRs on the flexibility of the algorithm. Specifically, we verify the revenue of the algorithm, the revenue cost ratio, and the virtual network acceptance rate.

C. Simulation Results and Performance Analysis

We first compare the long-term average revenue of the four algorithms, and the result is shown in Fig. 4. In the early stage of VNR, our algorithm performance is slightly worse than the other three algorithms. However, in the middle and late stages, our algorithm has a higher average revenue, indicating that our algorithm allocates physical network resources

TABLE III
ALGORITHM IDEA DESCRIPTION

Algorithm	Description
Our algorithm	The paper combines GCNN with RL algorithm, establishes a customized fitness matrix and fitness value, which aims to improve resource utilization, and improves the efficiency of the algorithm by model parallel training.
NodeRank [15]	Sort physical nodes according to the importance of the nodes, and use the breadth-first search strategy for virtual link mapping.
MCST-VNE [22]	The virtual node mapping is modeled as a Markov decision process, and MCST algorithm is used to specify the node mapping strategy.
GCN-VNE [28]	A VNE algorithm based on DRL and GCNN, which trains the model through a designed multi-objective reward function.

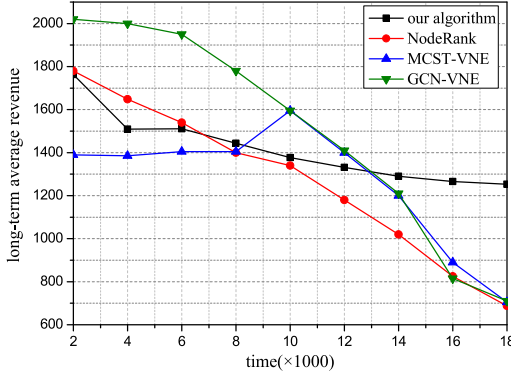


Fig. 4. Comparison with the long-term average revenue of other algorithms.

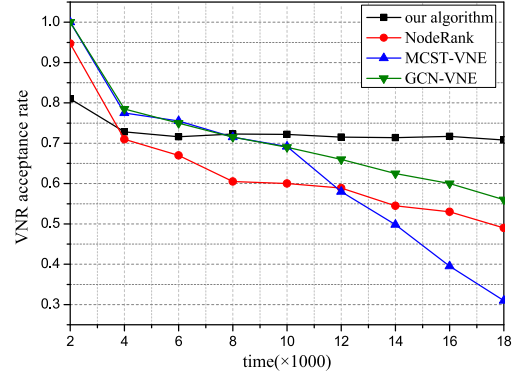


Fig. 5. Comparison with the VNR acceptance rate of other algorithms.

more reasonably, so that more resource consumption revenue can be obtained. From the experimental results, in the later stage of VNE, the long-term average revenue of our algorithm is 38.8%, 22.5%, and 24% higher than the other three algorithms, respectively. With the increase of time, the number of VNRs arriving at the physical network is increasing. The revenue calculation of VNE is shown in (9), which is determined by the consumption of physical resources. It should be noted that we calculate the long-term average revenue of VNE at time t rather than the cumulative revenue. Therefore, when the number of VNRs is increasing, the number of physical network resources consumed is increasing, and the remaining available resources of underlying network are decreasing. At this time, the scale of VNRs it can carry becomes smaller, and the revenue of resource consumption will be correspondingly reduced. It can be seen from Fig. 4 that the trend of the curve is decreasing with time, so it is in line with the actual situation.

We also compare the differences between the four algorithms in terms of virtual network acceptance rate, as shown in Fig. 5. The change curve of the VNR acceptance rate is similar to the revenue of VNE, but overall, the performance of our algorithm is better than the other three algorithms. After calculation, the acceptance rate of our algorithm is 15.2%, 14.5%, and 2.7% higher than the other three algorithms. An important reason is that we are concerned that VNE is a dynamic process. When selecting physical nodes for virtual nodes, we select those physical nodes with larger available resource capacity, so that the remaining physical resource capacity can also be used to carry other virtual nodes. On the one hand, it avoids

resource fragmentation, and on the other hand, it improves the acceptance rate of VNRs. The acceptance rate of VNRs will continue to decrease over time. This is because physical network resources are continuously consumed, and its remaining resources can only carry a small portion of VNRs, so the acceptance rate of VNRs will decrease. Therefore, the changing trend of VNR acceptance rate is explainable and reasonable.

Generally speaking, our algorithm has better stability, and the performance of the other three algorithms has changed a lot. On the one hand, they give priority to physical nodes and physical links with large resource capacity for embedding, and ignore the reasonable degree of resource allocation. In addition, NodeRank is a VNE algorithm based on the ranking of node importance, which only considers the local importance of node attributes, and the flexibility of the algorithm will be limited by the manual physical node selection rules. It shows that the performance of VNE based on ML is better than VNE based on heuristics. The MCST-VNE algorithm and GCN-VNE algorithm are based on the ML algorithm, but they both ignore the feature that VNE is dynamic, which may cause waste in allocating physical resources for VNRs, which is the main reason that their performance is not optimal.

We try to verify the flexibility of the algorithm by changing the resource attributes of physical network or VNRs. Since the cost and revenue of VNE are closely related to nodes resources and links resources, in the parameter setting part, we set the same capacity range for CPU resources and bandwidth resources. So we use the method of changing the CPU

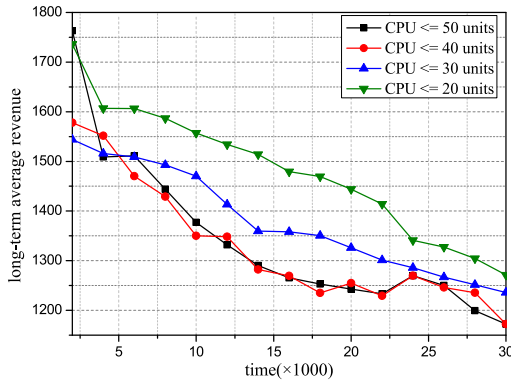


Fig. 6. Revenue test under different CPU resource requirements.

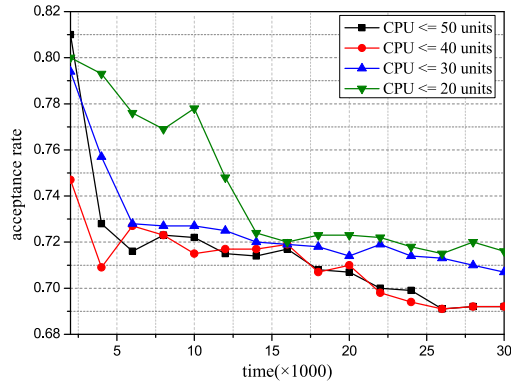


Fig. 7. Acceptance rate test under different CPU resource requirements.

resource requirement of the virtual node and the fixed bandwidth resource requirement to explore the impact of the change of the virtual network resource requirement attribute on the algorithm. The experimental results of algorithm flexibility verification are shown in Figs. 6–8.

Fig. 6 shows the impact of changes in CPU resource requirements on long-term average revenue. Overall, the average revenue of the algorithm shows a downward trend under four different CPU resource requirements, which is caused by the decrease of available resources. A noticeable turning point is that when the CPU resource demand is less than or equal to 40 units, the long-term average revenue is the lowest. When the demand for CPU resources decreases, the long-term average revenue begins to rise again. It can be analyzed that when the CPU resource demand of the virtual node is less than or equal to 50 units, although the resource consumption of the virtual node is large, it can also bring higher revenue to the physical network. When the CPU resource requirement of virtual nodes is less than or equal to 30 or 20 units, although each VNR embedded in the physical network has low revenue, they can make up for the loss of revenue by increasing the number of VNE.

Figs. 7 and 8 show the results of the change of VNR acceptance rate and revenue cost ratio with the change of virtual network resource demand, respectively. A similar turning point occurs when the CPU resource requirement of virtual nodes is less than or equal to 40 units. The change curve of the revenue cost ratio is different from the other two. This is because

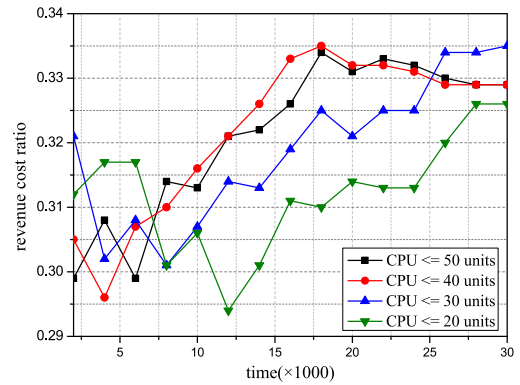


Fig. 8. Revenue cost ratio test under different CPU resource requirements.

the revenue cost ratio is not directly related to the size of the physical network resource capacity. It reflects the degree of resource utilization of the physical network and the profitability of the InPs. It can be inferred from the above experiments that changes in virtual link resource requirements can also produce similar effects. This shows that it is effective and realistic to explore the performance changes of the algorithm by changing the resource attributes of the physical network or virtual network. Therefore, our algorithm has good flexibility.

VII. CONCLUSION

The VNE algorithm still faces several problems, including unreasonable constraints and goal setting, ignoring the time feasibility of the algorithm, and not paying attention to the dynamic of the VNE process, which lead to the disadvantage of low flexibility of the existing VNE algorithm. This article proposes a dynamic VNE algorithm based on RL and GCNN, taking into account the dynamics of VNE and the basic performance of the algorithm. We combine GCNN with the RL algorithm, based on a self-defined fitness matrix and fitness value, to achieve an efficient dynamic VNE algorithm. This algorithm effectively reduces the degree of resource fragmentation by remapping VNRs. In RL, we use asynchronous advantage actor-critic algorithm for parallel training of the agent, which mainly includes key steps such as feature extraction, strategy generation, and model training. In experimental phase, we first compare the proposed algorithm with other three representative algorithms. Afterwards, by changing the resource attributes of physical network and virtual network, it is more comprehensively demonstrate that the algorithm not only has satisfactory basic VNE performance, but also has good flexibility.

REFERENCES

- [1] A. M. Alberti, G. D. Scarpioni, V. J. Magalhães, A. S. Cerqueira, J. J. P. C. Rodrigues, and R. D. R. Righi, "Advancing NovaGenesis architecture towards future Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 1, pp. 215–229, Feb. 2019.
- [2] P. Zhang, C. Wang, G. S. Aujla, N. Kumar, and M. Guizani, "IoV sCENARIO: Implementation of a bandwidth aware algorithm in wireless network communication mode," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 15774–15785, Dec. 2020.

- [3] S. Cherrared, S. Imadali, E. Fabre, G. Gößler, and I. G. B. Yahia, "A survey of fault management in network virtualization environments: Challenges and solutions," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 4, pp. 1537–1551, Dec. 2019.
- [4] S. Nazemi, K. K. Leung, and A. Swami, "Distributed optimization framework for in-network data processing," *IEEE/ACM Trans. Netw.*, vol. 27, no. 6, pp. 2432–2443, Dec. 2019.
- [5] P. Zhang, X. Pang, Y. Bi, H. Yao, H. Pan, and N. Kumar, "DSCD: Delay sensitive cross-domain virtual network embedding algorithm," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2913–2925, Oct.–Dec. 2020.
- [6] H. Cao, H. Zhu, and L. Yang, "Collaborative attributes and resources for single-stage virtual network mapping in network virtualization," *J. Commun. Netw.*, vol. 22, no. 1, pp. 61–71, Feb. 2020.
- [7] R. Chai, D. Xie, L. Luo, and Q. Chen, "Multi-objective optimization-based virtual network embedding algorithm for software-defined networking," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 1, pp. 532–546, Mar. 2020.
- [8] M. Lu, Y. Gu, and D. Xie, "A dynamic and collaborative multi-layer virtual network embedding algorithm in SDN based on reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2305–2317, Dec. 2020.
- [9] P. Zhang, C. Wang, C. Jiang, and A. Benslimane, "Security-aware virtual network embedding algorithm based on reinforcement learning," *IEEE Trans. Netw. Sci. Eng.*, early access, May 19, 2020, doi: [10.1109/TNSE.2020.2995863](https://doi.org/10.1109/TNSE.2020.2995863).
- [10] H. Afifi and H. Karl, "Reinforcement learning for virtual network embedding in wireless sensor networks," in *Proc. 16th Int. Conf. Wireless Mobile Comput. Netw. Commun. (WiMob)*, Thessaloniki, Greece, 2020, pp. 123–128.
- [11] I. Budhiraja, N. Kumar, and S. Tyagi, "Deep-reinforcement-learning-based proportional fair scheduling control scheme for underlay D2D communication," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3143–3156, Mar. 2021.
- [12] Y. Li, R. Chen, Y. Zhang, and H. Li, "A CNN-GCN framework for multi-label aerial image scene classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Waikoloa, HI, USA, 2020, pp. 1353–1356.
- [13] P. Zhang, C. Wang, C. Jiang, and Z. Han, "Deep reinforcement learning assisted federated learning algorithm for data management of IIoT," *IEEE Trans. Ind. Informat.*, early access, Mar. 8, 2021, doi: [10.1109/TII.2021.3064351](https://doi.org/10.1109/TII.2021.3064351).
- [14] M. Chowdhury, M. R. Rahman, and R. Boutaba, "ViNEYard: Virtual network embedding algorithms with coordinated node and link mapping," *IEEE/ACM Trans. Netw.*, vol. 20, no. 1, pp. 206–219, Feb. 2012.
- [15] X. Cheng *et al.*, "Virtual network embedding through topology-aware node ranking," *Comput. Commun. Rev.*, vol. 41, no. 2, pp. 38–47, Apr. 2011.
- [16] P. Zhang, H. Yao, and Y. Liu, "Virtual network embedding based on computing, network, and storage resource constraints," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3298–3304, Oct. 2018.
- [17] J. Rubio-Loyola, C. Aguilar-Fuster, G. Toscano-Pulido, R. Mijumbi, and J. Serrat-Fernandez, "Enhancing metaheuristic-based online embedding in network virtualization environments," *IEEE Trans. Netw. Service Manag.*, vol. 15, no. 1, pp. 200–216, Mar. 2018.
- [18] A. Song, W.-N. Chen, T. Gu, H. Yuan, S. Kwong, and J. Zhang, "Distributed virtual network embedding system with historical archives and set-based particle swarm optimization," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 51, no. 2, pp. 927–942, Feb. 2021.
- [19] P. Zhang, Y. Hong, X. Pang, and C. Jiang, "VNE-HPSO: Virtual network embedding algorithm based on hybrid particle swarm optimization," *IEEE Access*, vol. 8, pp. 213389–213400, 2020.
- [20] A. Song, W.-N. Chen, T. Gu, H. Zhang, and J. Zhang, "A constructive particle swarm optimizer for virtual network embedding," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 3, pp. 1406–1420, Jul.–Sep. 2020.
- [21] H. Yao, S. Ma, J. Wang, P. Zhang, C. Jiang, and S. Guo, "A continuous-decision virtual network embedding scheme relying on reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 2, pp. 864–875, Jun. 2020.
- [22] S. Haeri and L. Trajković, "Virtual network embedding via Monte Carlo tree search," *IEEE Trans. Cybern.*, vol. 48, no. 2, pp. 510–521, Feb. 2018.
- [23] Y. Yuan, Z. Tian, C. Wang, F. Zheng, and Y. Lv, "A Q-learning-based approach for virtual network embedding in data center," *Neural Comput. Appl.*, vol. 32, no. 7, pp. 1995–2004, Apr. 2020.
- [24] H. Yao, X. Chen, M. Li, P. Zhang, and L. Wang, "A novel reinforcement learning algorithm for virtual network embedding," *Neurocomputing*, vol. 284, pp. 1–9, Apr. 2018.
- [25] M. Dolati, S. B. Hassanpour, M. Ghaderi, and A. Khonsari, "DeepViNE: Virtual network embedding with deep reinforcement learning," in *Proc. IEEE INFOCOM Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Paris, France, 2019, pp. 879–885.
- [26] C. Wang, R. S. Batth, P. Zhang, G. S. Aujla, Y. Duan, and L. Ren, "VNE solution for network differentiated QoS and security requirements: From the perspective of deep reinforcement learning," *Computing*, vol. 103, no. 6, pp. 1061–1083, 2021.
- [27] F. Habibi, M. Dolati, A. Khonsari, and M. Ghaderi, "Accelerating virtual network embedding with graph neural networks," in *Proc. 16th Int. Conf. Netw. Service Manag. (CNSM)*, Izmir, Turkey, 2020, pp. 1–9.
- [28] Z. Yan, J. Ge, Y. Wu, L. Li, and T. Li, "Automatic virtual network embedding: A deep reinforcement learning approach with graph convolutional networks," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 6, pp. 1040–1057, Jun. 2020.
- [29] C. Zhao and B. Parhami, "Virtual network embedding through graph eigenspace alignment," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 2, pp. 632–646, Jun. 2019.
- [30] T. Ghazar and N. Samaan, "A hierarchical approach for efficient virtual network embedding based on exact subgraph matching," in *Proc. IEEE Global Telecommun. Conf. (GLOBECOM)*, Houston, TX, USA, 2011, pp. 1–6.
- [31] C. K. Dehury and P. K. Sahoo, "DYVINE: Fitness-based dynamic virtual network embedding in cloud computing," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1029–1045, May 2019.
- [32] M. Lu, Y. Gu, and D. Xie, "A dynamic and collaborative multi-layer virtual network embedding algorithm in SDN based on reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2305–2317, Dec. 2020.
- [33] H. Cao, S. Wu, G. S. Aujla, Q. Wang, L. Yang, and H. Zhu, "Dynamic embedding and quality of service-driven adjustment for cloud networks," *IEEE Trans. Ind. Informat.*, vol. 16, no. 2, pp. 1406–1416, Feb. 2020.
- [34] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 3844–3852.